

**МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ ЭЛЕКТРОТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ
“ЛЭТИ” ИМ.В.И.УЛЬЯНОВА (ЛЕНИНА)»
КАФЕДРА МОЭВМ**

**ОТЧЕТ
по лабораторно-практической работе № 4
«Обработка исключений»
по дисциплине «Объектно - ориентированное программирование на
языке Java»**

Выполнил: Шарапов И.Д.

Факультет: КТИ

Группа: №3312

Подпись преподавателя: _____

Санкт-Петербург

2024

Содержание

Цель работы	3
Перечень ситуаций, которые контролируются с помощью исключений	3
Пример работы обработчиков ситуаций.....	4
Текст программы	5
Приложение	10

Цель работы

Знакомство с механизмом обработки исключений в языке Java.

Перечень ситуаций, которые контролируются с помощью исключений

- Пустое поле поиска: Если пользователь пытается выполнить поиск, не введя текст в поле поиска, вызывается пользовательское исключение *EmptySearchException*. Это предотвращает выполнение поиска без запроса и предупреждает пользователя об ошибке.
- Отсутствие значения в поле поиска¹: Если поле поиска содержит значение *null*, вызывается стандартное исключение *NullPointerException*. Это исключение гарантирует, что поиск не будет выполнен, если значение отсутствует, и отображает пользователю сообщение о необходимости ввода текста.
- Отсутствие совпадений при поиске: Если ни одна строка в таблице не содержит искомый текст, пользователю показывается информационное сообщение о том, что совпадения не найдены. Это не является исключением, но обеспечивает корректную обработку ситуации отсутствия результатов поиска.

¹ Исключение *NullPointerException* возникает, если поле поиска само по себе имеет значение *null*, что является нештатной ситуацией, возможно, из-за логической ошибки или непредвиденного поведения в программе. Это исключение применяется для контроля за более серьёзной ошибкой в коде, чем ошибка ввода пользователя, и указывает на возможность неисправности программы.

Пример работы обработчиков ситуаций

ГAI System

Добавить Редактировать Удалить Загрузить Сохранить

ФИО водителя	Номер машины	Дата нарушения	Тип нарушения
Иванов Иван Иванович	A123BC77	15.03.2024	Превышение скорости
Петров Петр Петрович	B456MN77	20.07.2023	Проезд на красный свет
Смирнова Анна Сергеевна	C789OP77	05.05.2024	Нарушение парковки
Кузнецова Мария Александровна	D123EK77		Отсутствие страховки
Соколов Сергей Викторович	E456TP77		Разворот в неподобающем месте

Ошибка

Поле поиска не должно быть пустым

OK

По имени

Поиск

Рисунок 1 – Поиск пустой строки

ГAI System

Добавить Редактировать Удалить Загрузить Сохранить

ФИО водителя	Номер машины	Дата нарушения	Тип нарушения
Иванов Иван Иванович	A123BC77	15.03.2024	Превышение скорости
Петров Петр Петрович	B456MN77	20.07.2023	Проезд на красный свет
Смирнова Анна Сергеевна	C789OP77	05.05.2024	Нарушение парковки
Кузнецова Мария Александровна	D123EK77	12.12.2023	Отсутствие страховки
Соколов Сергей Викторович	E456TP77	22.02.2024	Разворот в неподобающем месте

По имени

ВИЧ

Поиск

Рисунок 2 – Поиск по подстроке в имени

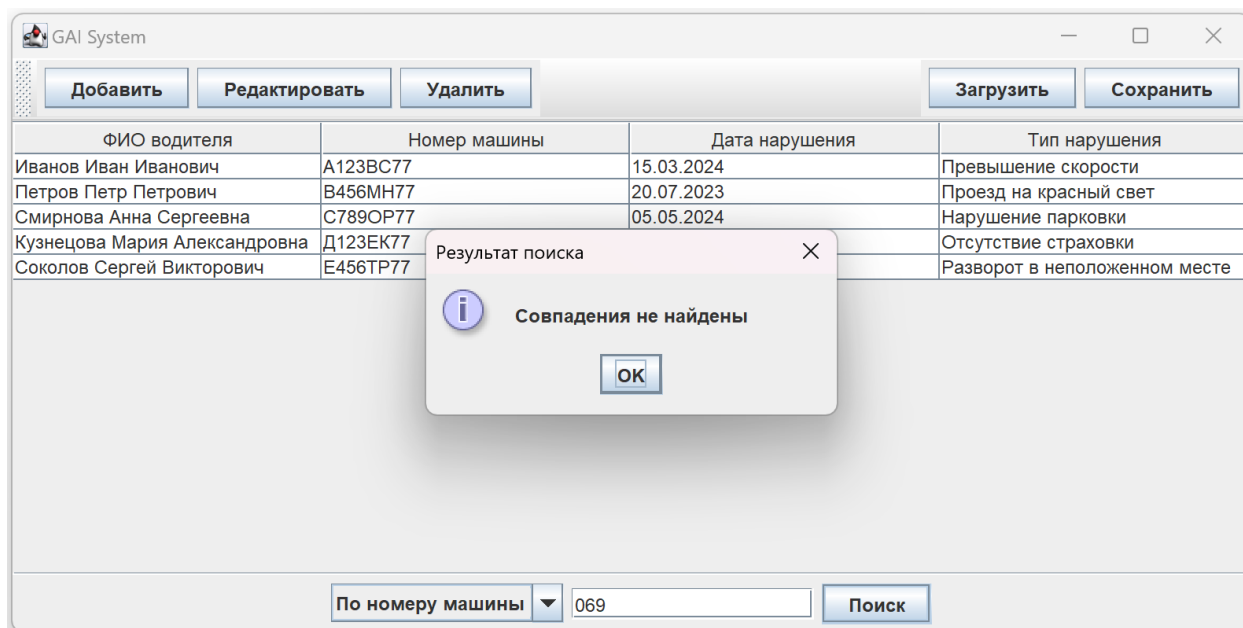


Рисунок 3 – Поиск не нашёл подстроки

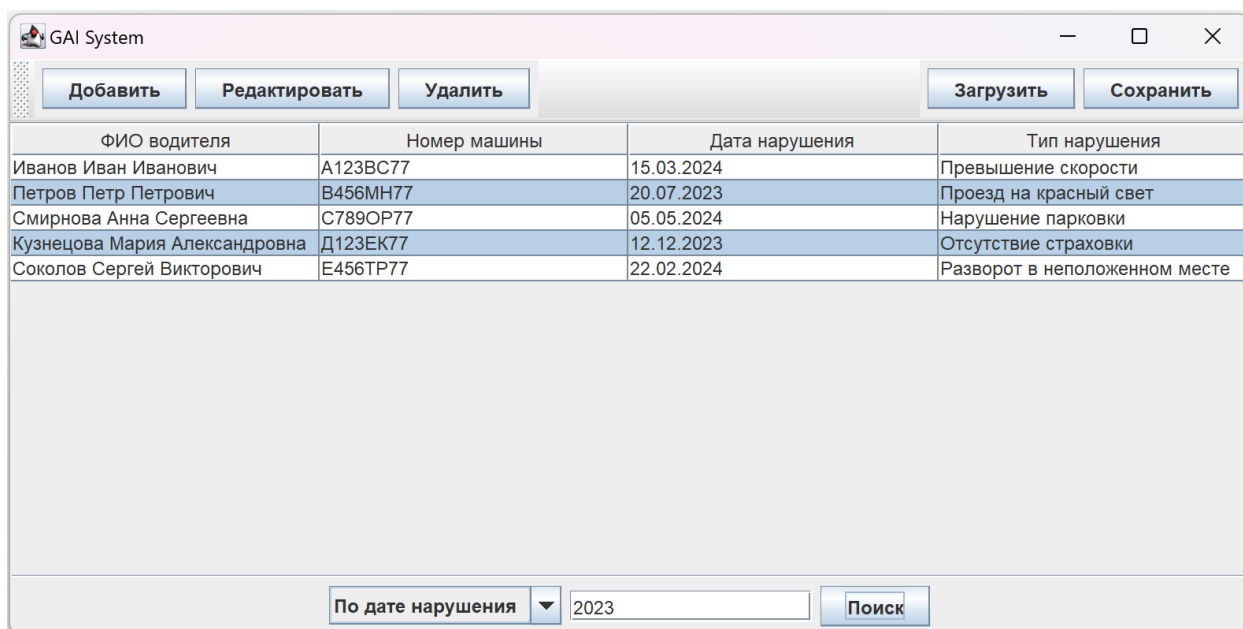


Рисунок 4 – Поиск по дате

Текст программы

Класс Main.java

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.io.*;

import Exceptions.EmptySearchException;

/**
 * Программа для работы с данными о водителях и их нарушениях.
 * Содержит функции добавления, редактирования, удаления записей, а также
 * сохранения и загрузки данных в файл.
 */
```

```

*
* @author Шарапов Иван 3312
* @version 1.2
*/
public class Main {
    private JFrame mainFrame;
    private DefaultTableModel tableModel;
    private JTable dataTable;
    private JButton addDriverButton, editDriverButton, deleteDriverButton,
loadDriverButton, saveDriverButton;
    private JTextField searchField;
    private JComboBox<String> searchTypeComboBox;

    /**
     * Конструктор класса Main.
     * Инициализирует основное окно приложения для работы с данными.
     */
    public Main() {
        // Конструктор по умолчанию, который инициализирует класс Main.
    }

    /**
     * Метод для создания и отображения главного окна программы.
     * Включает создание таблицы, панели инструментов с кнопками и панель
поиска.
     */
    public void show() {
        // Создание основного окна приложения
        mainFrame = new JFrame("GAI System");
        mainFrame.setSize(800, 400);
        mainFrame.setLocation(100, 100);
        mainFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        // Создание кнопок для управления записями
        addDriverButton = new JButton("Добавить");
        editDriverButton = new JButton("Редактировать");
        deleteDriverButton = new JButton("Удалить");
        loadDriverButton = new JButton("Загрузить");
        saveDriverButton = new JButton("Сохранить");

        // Панель инструментов, которая содержит кнопки
        JToolBar toolBar = new JToolBar("Toolbar");
        toolBar.setLayout(new BorderLayout()); // Устанавливаем BorderLayout
для панели инструментов

        // Панель с кнопками управления (добавление, редактирование,
удаление)
        JPanel leftPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
        leftPanel.add(addDriverButton);
        leftPanel.add(editDriverButton);
        leftPanel.add(deleteDriverButton);
        toolBar.add(leftPanel, BorderLayout.WEST); // Размещаем в левой части
панели

        // Панель с кнопками сохранения и загрузки
        JPanel rightPanel = new JPanel(new FlowLayout(FlowLayout.RIGHT));
        rightPanel.add(loadDriverButton);
        rightPanel.add(saveDriverButton);
        toolBar.add(rightPanel, BorderLayout.EAST); // Размещаем в правой
части панели

        mainFrame.setLayout(new BorderLayout());
        mainFrame.add(toolBar, BorderLayout.NORTH); // Размещение панели
инструментов сверху

        // Создание таблицы для отображения данных
        String[] columns = {"ФИО водителя", "Номер машины", "Дата нарушения",
"Тип нарушения"};
        String[][] data = {
            {"Иванов Иван Иванович", "A123BC77", "15.03.2024",
"Превышение скорости"},
            {"Петров Петр Петрович", "B456MN77", "20.07.2023", "Проезд на
красный свет"},
            {"Смирнова Анна Сергеевна", "C789OP77", "05.05.2024",
"Нарушение парковки"},
            {"Кузнецова Мария Александровна", "D123EK77", "12.12.2023",
"Отсутствие страховки"},
            {"Соколов Сергей Викторович", "E456TP77", "22.02.2024",

```

```

"Разворот в неподленном месте"}
};
tableModel = new DefaultTableModel(data, columns);
dataTable = new JTable(tableModel); // Таблица, которая использует
данные tableModel
JScrollPane scrollPane = new JScrollPane(dataTable); // Добавляем
прокрутку для таблицы
mainFrame.add(scrollPane, BorderLayout.CENTER); // Размещаем таблицу
в центре окна

// Элементы поиска: поле ввода и кнопка "Поиск"
searchTypeComboBox = new JComboBox<>(new String[]{"По имени", "По
номеру машины", "По дате нарушения", "По типу нарушения"});
searchField = new JTextField(15);
JButton searchButton = new JButton("Поиск");

JPanel searchPanel = new JPanel();
searchPanel.add(searchTypeComboBox);
searchPanel.add(searchField);
searchPanel.add(searchButton);
mainFrame.add(searchPanel, BorderLayout.SOUTH); // Панель поиска
размещается внизу

// Добавляем действия для кнопок

// "Добавить" — действие для добавления новой записи
addDriverButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        JOptionPane.showMessageDialog(mainFrame, "Добавление новой
записи");
    }
});

// "Редактировать" — действие для редактирования выбранной записи
editDriverButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        JOptionPane.showMessageDialog(mainFrame, "Редактирование
выбранной записи");
    }
});

// "Удалить" — действие для удаления выбранной записи
deleteDriverButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        JOptionPane.showMessageDialog(mainFrame, "Удаление выбранной
записи");
    }
});

// "Загрузить" — открывает диалоговое окно для выбора файла и
загружает данные в таблицу
loadDriverButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        JFileChooser fileChooser = new JFileChooser();
        fileChooser.setCurrentDirectory(new
File(System.getProperty("user.dir"))); // Устанавливаем корневую директорию
проекта
        int returnValue = fileChooser.showOpenDialog(mainFrame); //
Показываем диалог открытия файла
        if (returnValue == JFileChooser.APPROVE_OPTION) { // Если
файл выбран
            File selectedFile = fileChooser.getSelectedFile(); //
Получаем выбранный файл
            loadDataFromFile(selectedFile); // Загружаем данные из
файла в таблицу
        }
    }
});

// "Сохранить" — открывает диалоговое окно для сохранения файла и
записывает данные таблицы в файл
saveDriverButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {

```

```

        JFileChooser fileChooser = new JFileChooser();
        fileChooser.setCurrentDirectory(new
File(System.getProperty("user.dir"))); // Устанавливаем корневую директорию
проекта
        int returnValue = fileChooser.showSaveDialog(mainFrame); //
Показываем диалог сохранения файла
        if (returnValue == JFileChooser.APPROVE_OPTION) { // Если
выбрано место для сохранения
            File selectedFile = fileChooser.getSelectedFile(); //
Получаем файл, который выбрал пользователь
            saveDataToFile(selectedFile); // Сохраняем данные таблицы
в этот файл
        }
    });

    // "Поиск" — выполняет поиск в таблице, по введённой строке
    searchButton.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            try {
                validateSearchField(searchField); // Проверка значений в
поле поиска
                performSearch(searchField.getText()); // Выполнение
поиска по введённому тексту
            } catch (NullPointerException ex) {
                JOptionPane.showMessageDialog(mainFrame, ex.getMessage(),
"Ошибка", JOptionPane.ERROR_MESSAGE);
            } catch (EmptySearchException ex) {
                JOptionPane.showMessageDialog(mainFrame, ex.getMessage(),
"Ошибка", JOptionPane.WARNING_MESSAGE);
            }
        }
    });

    mainFrame.setVisible(true); // Делаем главное окно видимым
}

/**
 * Сохраняет данные из таблицы в указанный файл.
 *
 * @param file файл, в который будут сохранены данные.
 */
private void saveDataToFile(File file) {
    try (BufferedWriter writer = new BufferedWriter(new
FileWriter(file))) {
        // Проходим по каждой строке таблицы
        for (int row = 0; row < tableModel.getRowCount(); row++) {
            for (int col = 0; col < tableModel.getColumnCount(); col++) {
                // Записываем значения ячеек таблицы в файл через
табуляцию
                writer.write(tableModel.getValueAt(row, col).toString() +
"\t");
            }
            writer.newLine(); // Переход на новую строку после каждой
записи
        }
        JOptionPane.showMessageDialog(mainFrame, "Данные успешно
сохранены!"); // Сообщение об успешном сохранении
    } catch (IOException e) {
        JOptionPane.showMessageDialog(mainFrame, "Ошибка при сохранении
данных."); // Сообщение об ошибке
    }
}

/**
 * Загружает данные из указанного файла в таблицу.
 *
 * @param file файл, из которого будут загружены данные.
 */
private void loadDataFromFile(File file) {
    try (BufferedReader reader = new BufferedReader(new
FileReader(file))) {
        String line;
        tableModel.setRowCount(0); // Очищаем текущие данные таблицы
перед загрузкой новых
        while ((line = reader.readLine()) != null) {
            // Разделяем строку по табуляциям, чтобы получить значения

```



```

для каждой колонки
        String[] rowData = line.split("\t");
        tableModel.addRow(rowData); // Добавляем новую строку в
таблицу
    }
    JOptionPane.showMessageDialog(mainFrame, "Данные успешно
загружены!"); // Сообщение об успешной загрузке
    } catch (IOException e) {
        JOptionPane.showMessageDialog(mainFrame, "Ошибка при загрузке
данных."); // Сообщение об ошибке
    }
}

/**
 * Возвращает индекс столбца для поиска на основе выбранного поля.
 *
 * @param selectedField Поле, выбранное пользователем в JComboBox для
поиска
 * @return Индекс столбца для поиска, или -1, если поле не распознано
 */
private int getColumnIndex(String selectedField) {
    switch (selectedField) {
        case "По имени":
            return 0; // Индекс столбца с ФИО водителя
        case "По номеру машины":
            return 1; // Индекс столбца с номером машины
        case "По дате нарушения":
            return 2; // Индекс столбца с датой нарушения
        case "По типу нарушения":
            return 3; // Индекс столбца с типом нарушения
        default:
            return -1;
    }
}

/**
 * Проверяет значение в поле поиска и генерирует исключения, если поле
пустое или содержит null.
 *
 * @param searchField Поле ввода текста для поиска
 * @throws EmptySearchException Если поле ввода пустое
 * @throws NullPointerException Если значение в поле null
 */
private void validateSearchField(JTextField searchField) throws
EmptySearchException, NullPointerException {
    String searchText = searchField.getText();
    if (searchText == null) {
        throw new NullPointerException("Поисковый запрос отсутствует");
    }
    if (searchText.isEmpty()) {
        throw new EmptySearchException();
    }
}

/**
 * Выполняет поиск по таблице и выделяет все строки, содержащие указанный
текст в выбранном поле, без учета регистра.
 *
 * @param query Строка для поиска
 */
private void performSearch(String query) {
    dataTable.clearSelection(); // Снимаем предыдущее выделение
    boolean found = false;

    // Приводим запрос к нижнему регистру
    String lowerCaseQuery = query.toLowerCase();

    // Получаем индекс столбца для поиска
    int columnIndex = getColumnIndex((String)
searchTypeComboBox.getSelectedItem());
    if (columnIndex == -1) {
        JOptionPane.showMessageDialog(mainFrame, "Некорректное поле для
поиска", "Ошибка", JOptionPane.ERROR_MESSAGE);
        return;
    }

    // Проходим по всем строкам, но только в выбранном столбце
    for (int i = 0; i < tableModel.getRowCount(); i++) {

```

```

        String cellValue = tableModel.getValueAt(i,
columnIndex).toString().toLowerCase();

        // Если значение ячейки содержит искомый текст без учета регистра
        if (cellValue.contains(lowerCaseQuery)) {
            dataTable.addRowSelectionInterval(i, i); // Выделяем строку
            found = true;
        }

        if (!found) {
            JOptionPane.showMessageDialog(mainFrame, "Совпадения не найдены",
"Результат поиска", JOptionPane.INFORMATION_MESSAGE);
        }
    }

    /**
     * Точка входа в приложение. Запускает метод show() для отображения GUI.
     * @param args Аргументы командной строки (не используются).
     */
    public static void main(String[] args) {
        new Main().show(); // Запуск приложения
    }
}

```

Класс EmptySearchException.java

```

package Exceptions;

public class EmptySearchException extends Exception {
    public EmptySearchException() {
        super("Поле поиска не должно быть пустым");
    }
}

```

Приложение

Ссылка на видео:

<https://rutube.ru/video/private/d3cecbea55ec7580cdea95cd375414f6/?p=biFRZqKI3zTlONtuQjW4pA>

Ссылка на репозиторий:

https://github.com/DexTver/OOP_ETU/tree/lab_04